

Deep Learning Lecture 7

Recurrent Neural Networks (RNNs) and Transformers

SLIDES BY KEVIN BRYSON

Overview

- We have seen fully connected feed-forward networks and these are useful for modelling certain systems but not for images.
- We can model images better using CNNs with kernels that are applied over entire images to detect patterns within them.
Essentially this is using weight sharing to detect common patterns.
- Recurrent Neural Networks have a similar idea of weight sharing but they do it in a different manner to analyse sequences ...

For example, consider the classical form of a dynamical system:

$$s_t = f(s_{t-1}; \theta),$$

where s_t is called the state of the system.

For time steps τ , the graph can be unfolded by applying the definition $\tau - 1$ times. For $\tau = 3$ time steps, we obtain $s_3 = f(s_2; \theta) = f(f(s_1; \theta); \theta)$.

Unfolding the equation by repeatedly applying the definition has yielded an expression that does not involve recurrence, which can now be represented by a traditional directed acyclic computational graph.

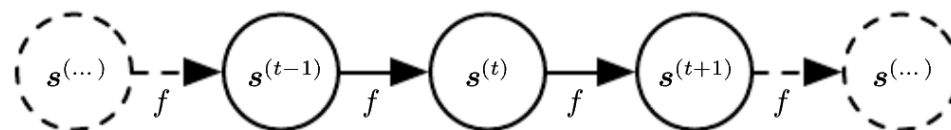
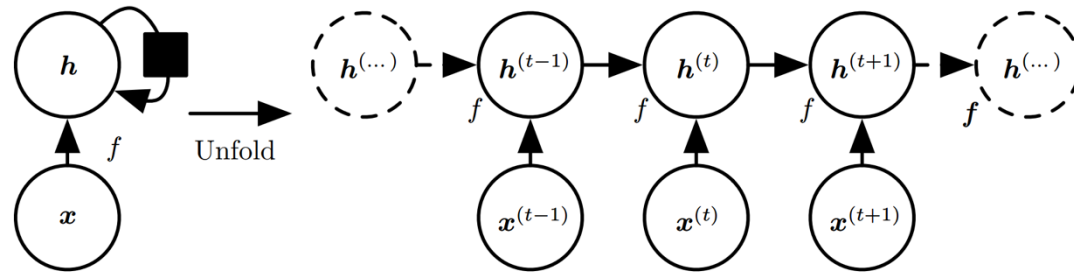


Figure 10.1: The classical dynamical system described by equation 10.1, illustrated as an unfolded computational graph. Each node represents the state at some time t , and the function f maps the state at t to the state at $t + 1$. The same parameters (the same value of θ used to parametrize f) are used for all time steps.

System with inputs

Many recurrent neural networks use this equation to define the values of their hidden units. To show that the state is the hidden units of the network, we rewrite it using the variable h to represent the state:

$$h(t) = f(h(t-1), x(t); \theta).$$



Typical RNNs will add extra architectural features such as output layers that read information out of the state h to make predictions

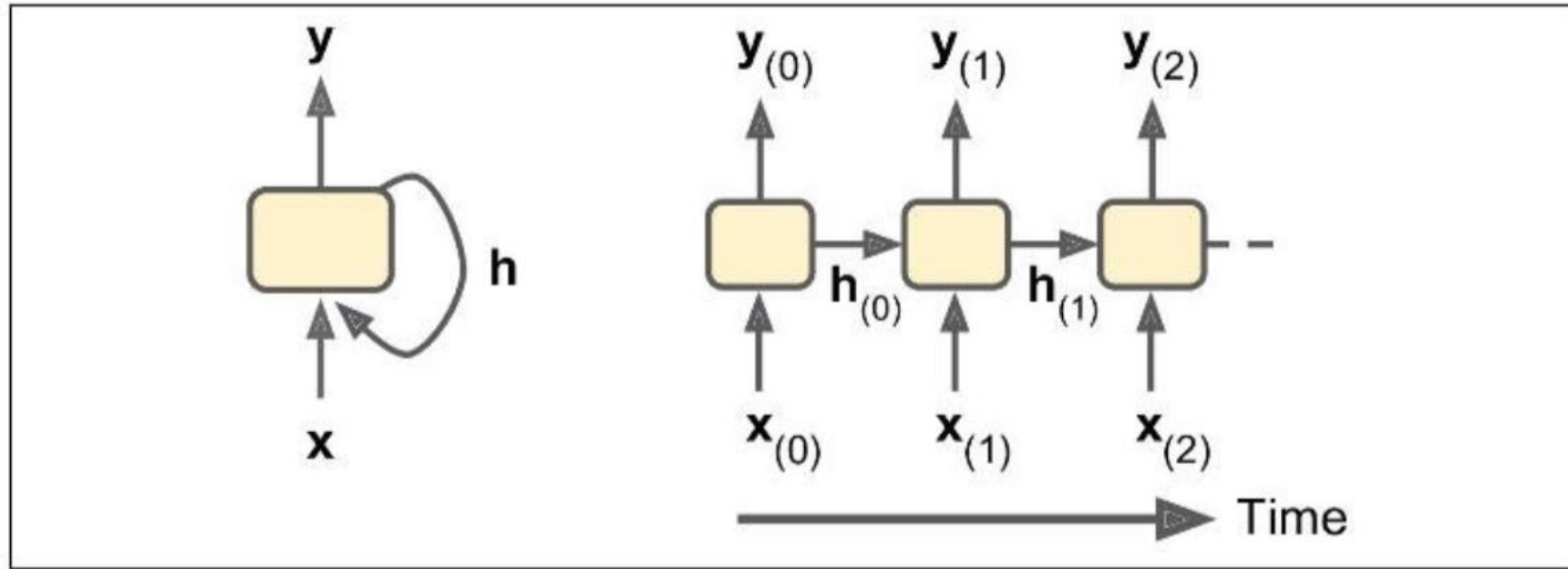


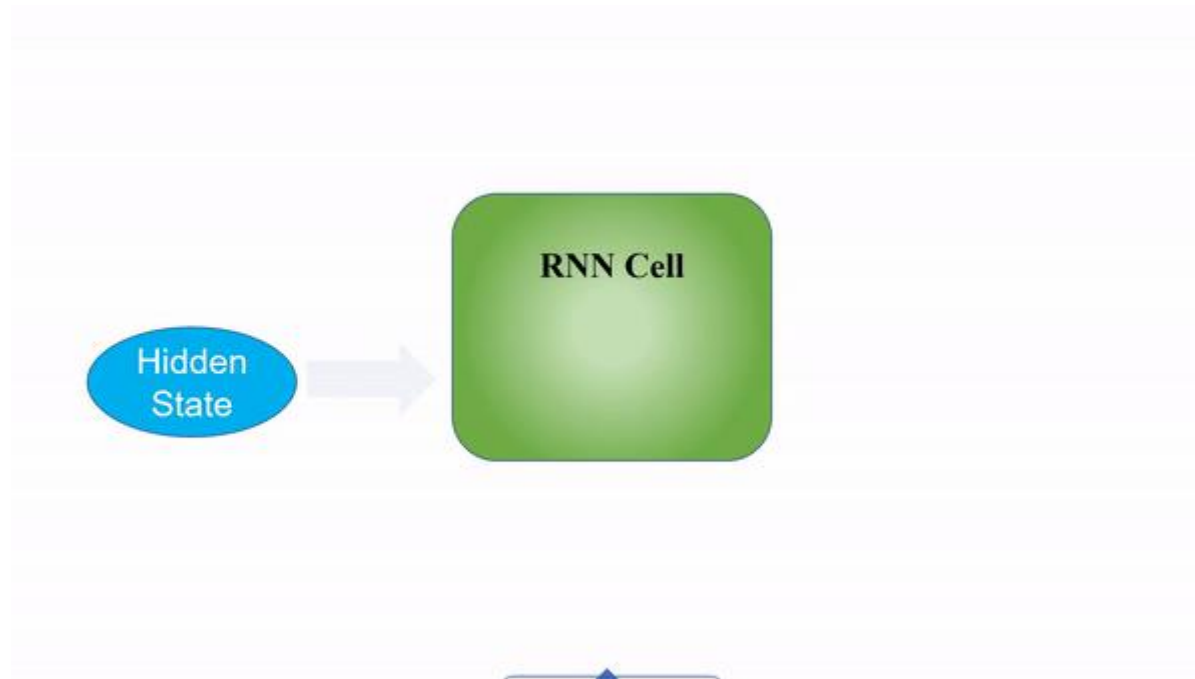
Figure 14-3. A cell's hidden state and its output may be different

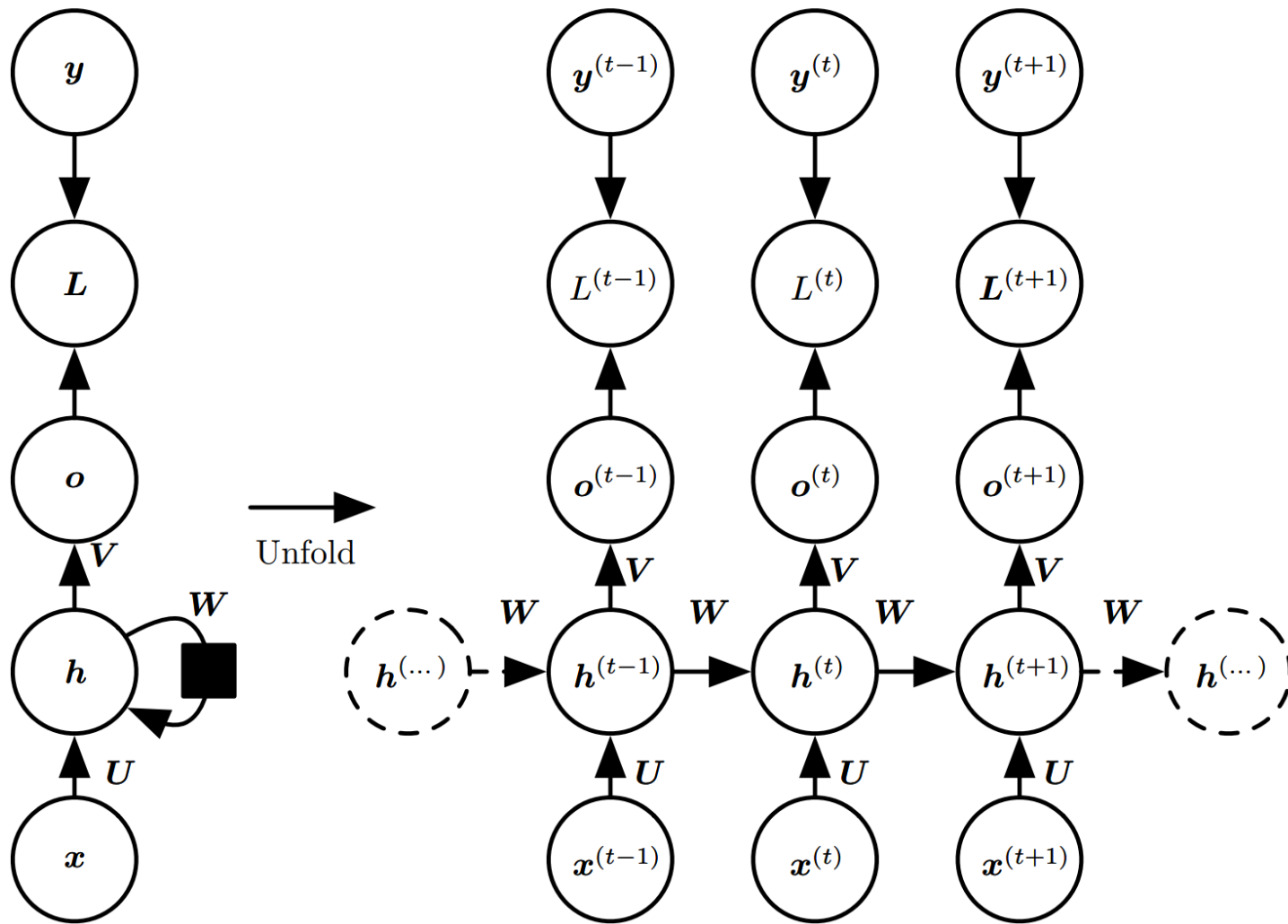
Unrolling the network, with outputs

The state at time t depends on the state at time $t-1$

Loop unrolling: turns recurrent net into feed-forward net with shared weights

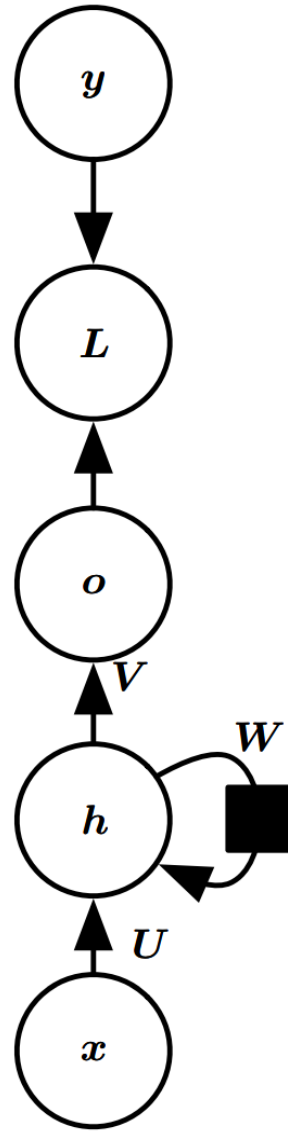
Animating the flow of an RNN





A typical
RNN
architecture

Forward
propagation
equations



$$a_t = W h_{t-1} + U x_t + b$$

$$h_t = \tanh(h_t)$$

$$o_t = \text{softmax}(V a_t + c)$$

$$L_t = \text{CE}(o_t, y_t)$$

Listing 6.21 Numpy implementation of a simple RNN

Number of timesteps in the input sequence

```
import numpy as np
```

```
timesteps = 100
```

```
input_features = 32
```

```
output_features = 64
```

Dimensionality of the input feature space

Dimensionality of the output feature space

Input data: random noise for the sake of the example

```
inputs = np.random.random((timesteps, input_features))
```

```
state_t = np.zeros((output_features,))
```

Initial state: an all-zero vector

```
W = np.random.random((output_features, input_features))
```

```
U = np.random.random((output_features, output_features))
```

```
b = np.random.random((output_features,))
```

Creates random weight matrices

```
successive_outputs = []
```

```
for input_t in inputs:
```

input_t is a vector of shape (input_features,).

```
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
```

```
    successive_outputs.append(output_t)
```

```
    state_t = output_t
```

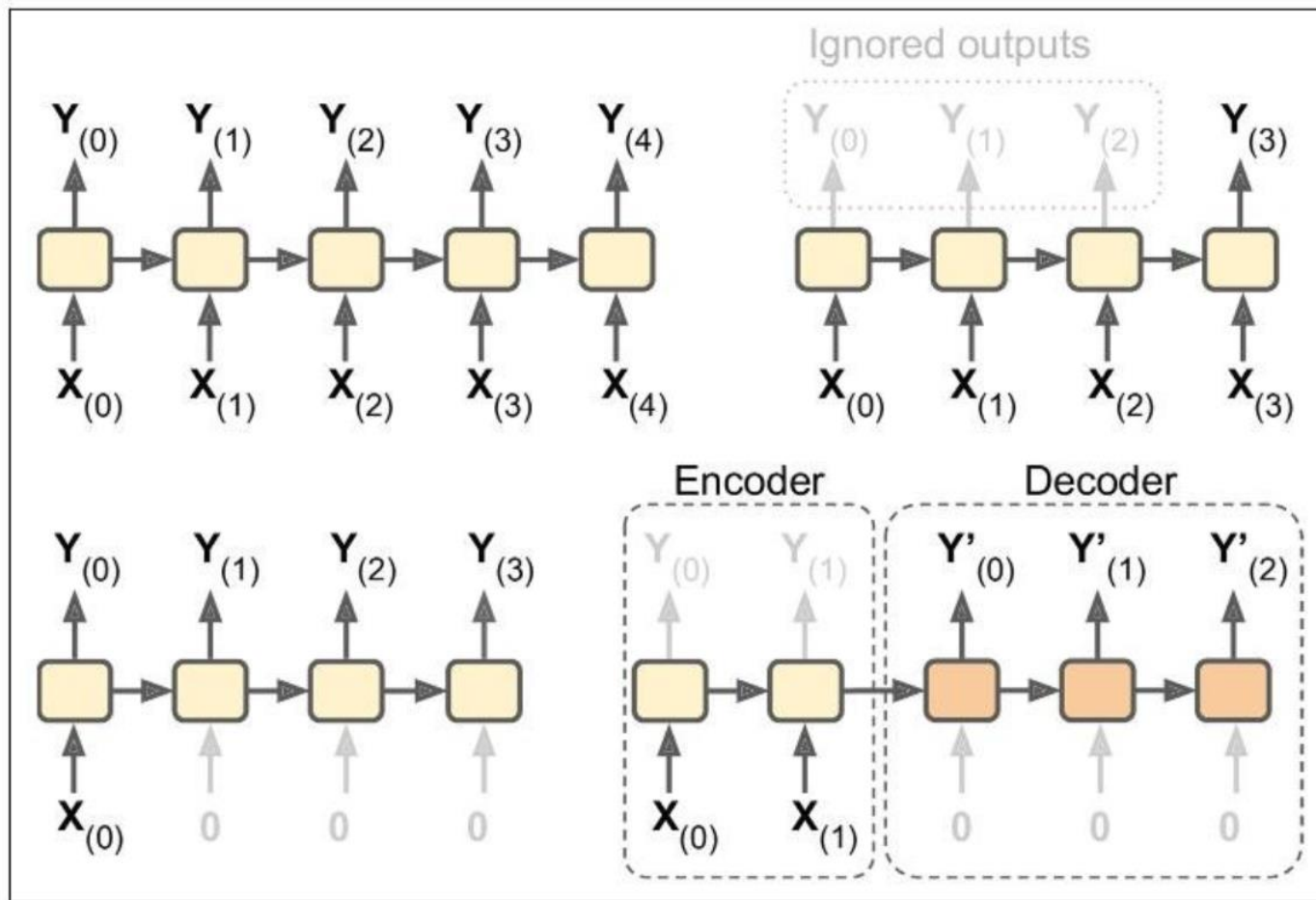
```
final_output_sequence = np.concatenate(successive_outputs, axis=0)
```

Stores this output in a list

The final output is a 2D tensor of shape (timesteps, output_features).

Combines the input with the current state (the previous output) to obtain the current output

Updates the state of the network for the next timestep



Input/Output Sequence options

Figure 14-4. Seq to seq (top left), seq to vector (top right), vector to seq (bottom left), delayed seq to seq (bottom right)

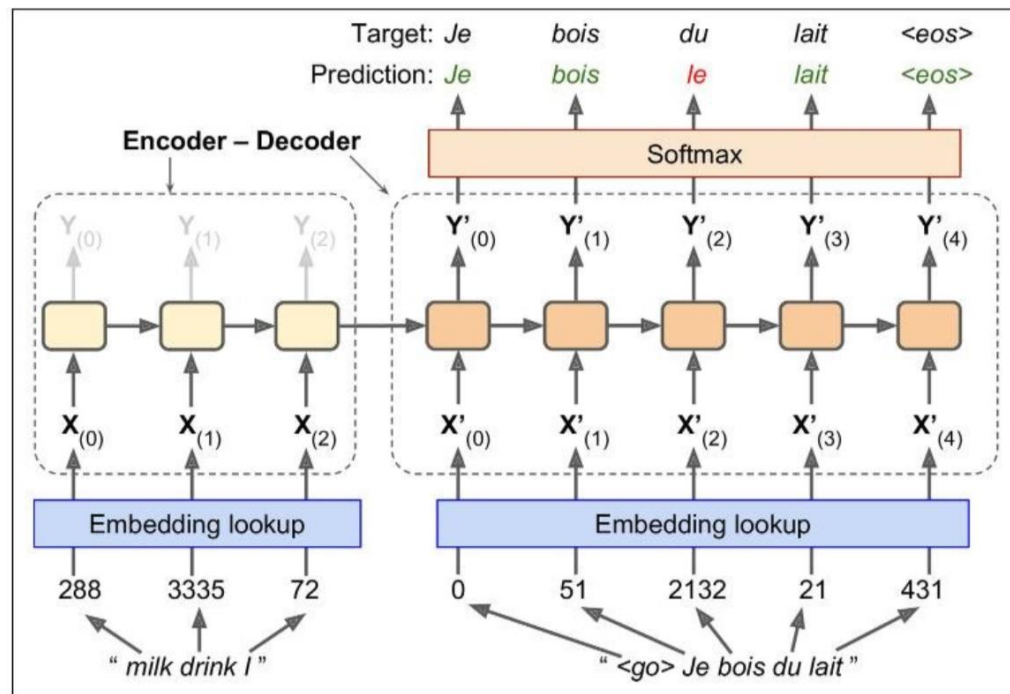


Figure 14-15. A simple machine translation model

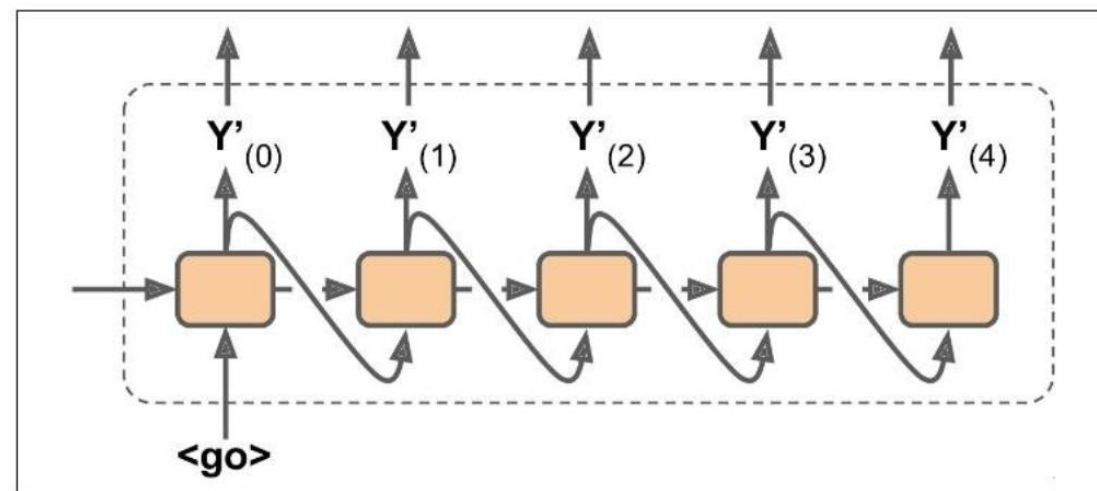
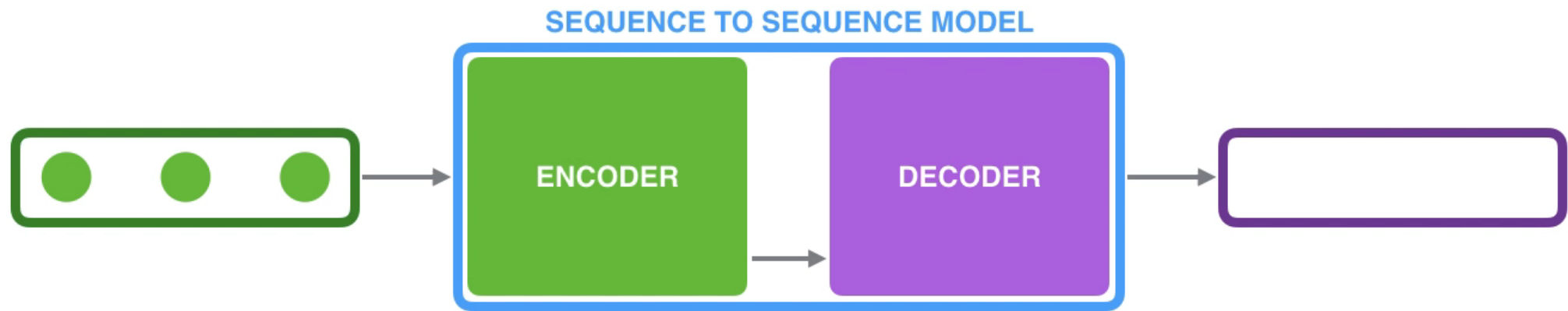


Figure 14-16. Feeding the previous output word as input at inference time

Encoder-Decoders for Machine translation

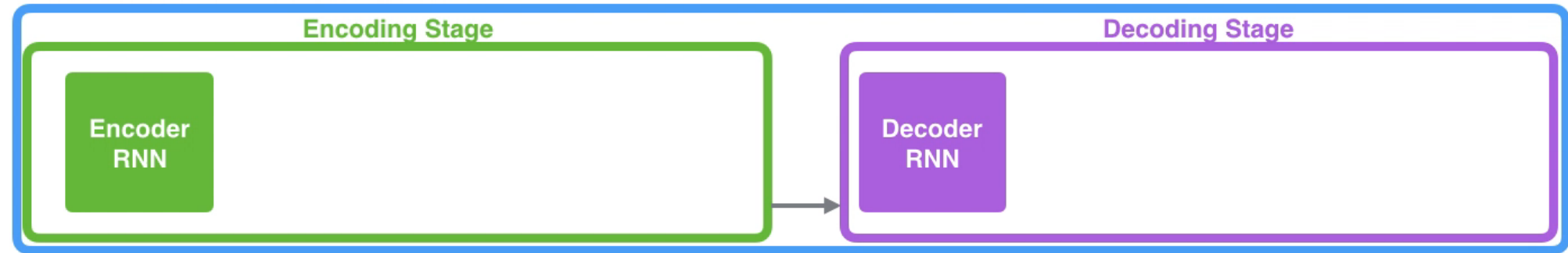


<https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

Unrolling the RNN over time

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL



Je

suis

étudiant

$$L = L(s_T(s_{T-1}(\dots s_{t+1}(s_t(\dots))))))$$

$$\frac{dL}{ds_t} = \frac{dL}{ds_T} \frac{ds_T}{ds_{T-1}} \dots \frac{ds_{t+1}}{ds_t}$$

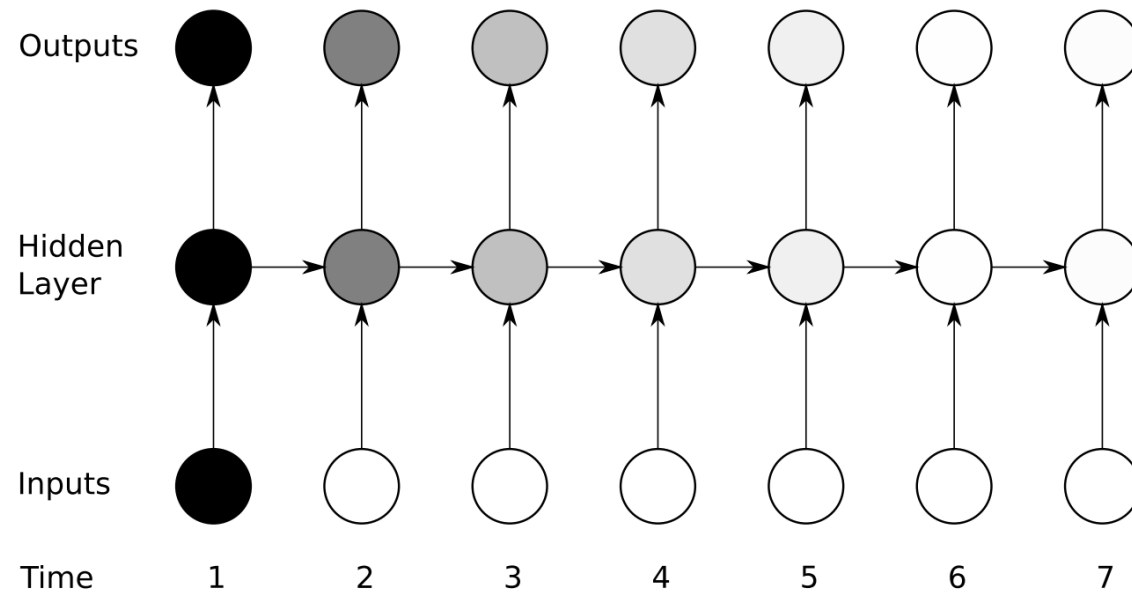
writing $\frac{ds_k}{ds_i} = \prod_{j=i}^{k-1} \frac{ds_{j+1}}{ds_j}$, then back-propagation of error in an RNN is:

$$\frac{dE_k}{dW} = \frac{dE_k}{d\hat{y}_k} \frac{d\hat{y}_k}{ds_k} \sum_{i=0}^k \left(\prod_{j=i}^{k-1} \frac{ds_{j+1}}{ds_j} \right) \frac{ds_i}{dW}$$

Problems:

- sing. values of Jacobians > 1 means gradients explode
- sing. values < 1 means gradients shrink & vanish
- random means variance grows exponentially

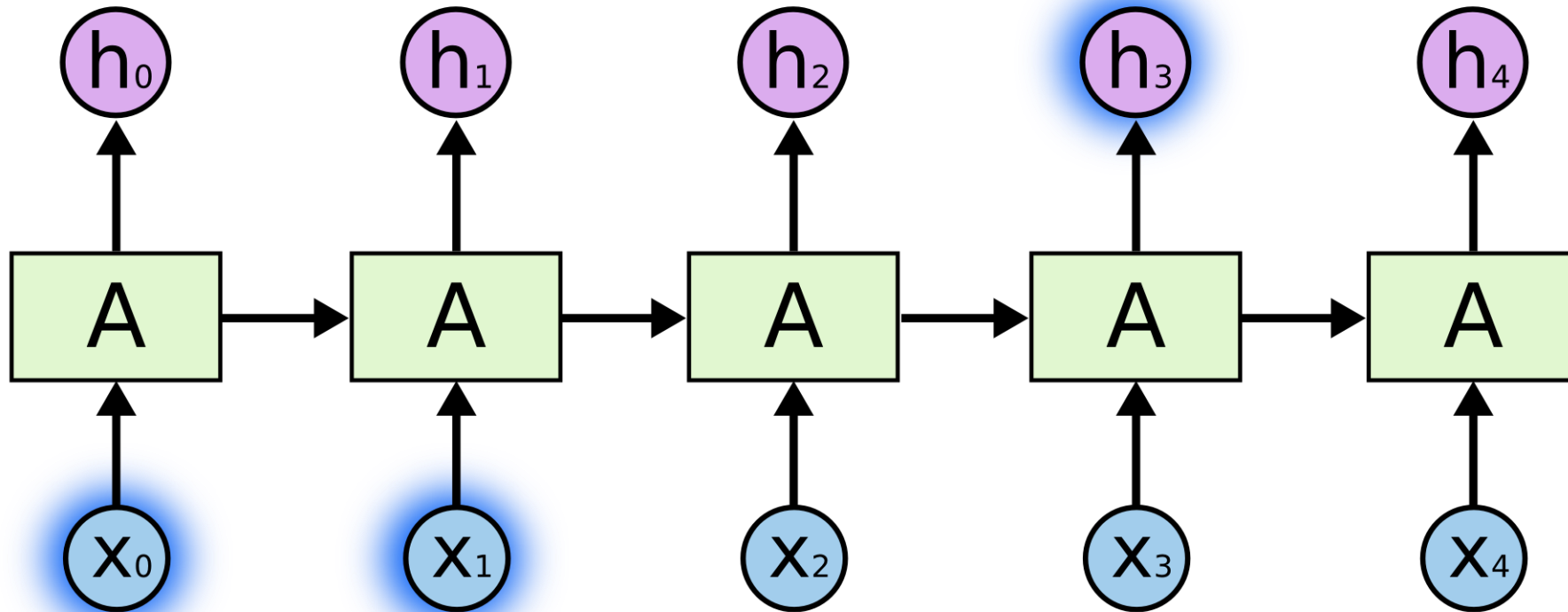
It's the same unit
repeated ... thus
exploding /
vanishing gradient



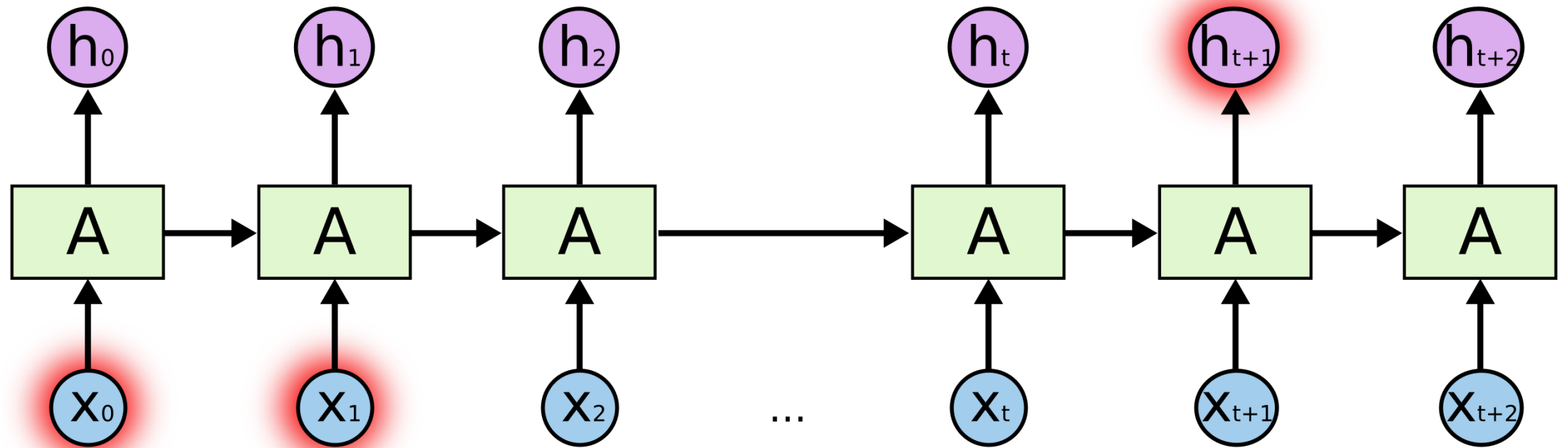
Vanishing Gradient

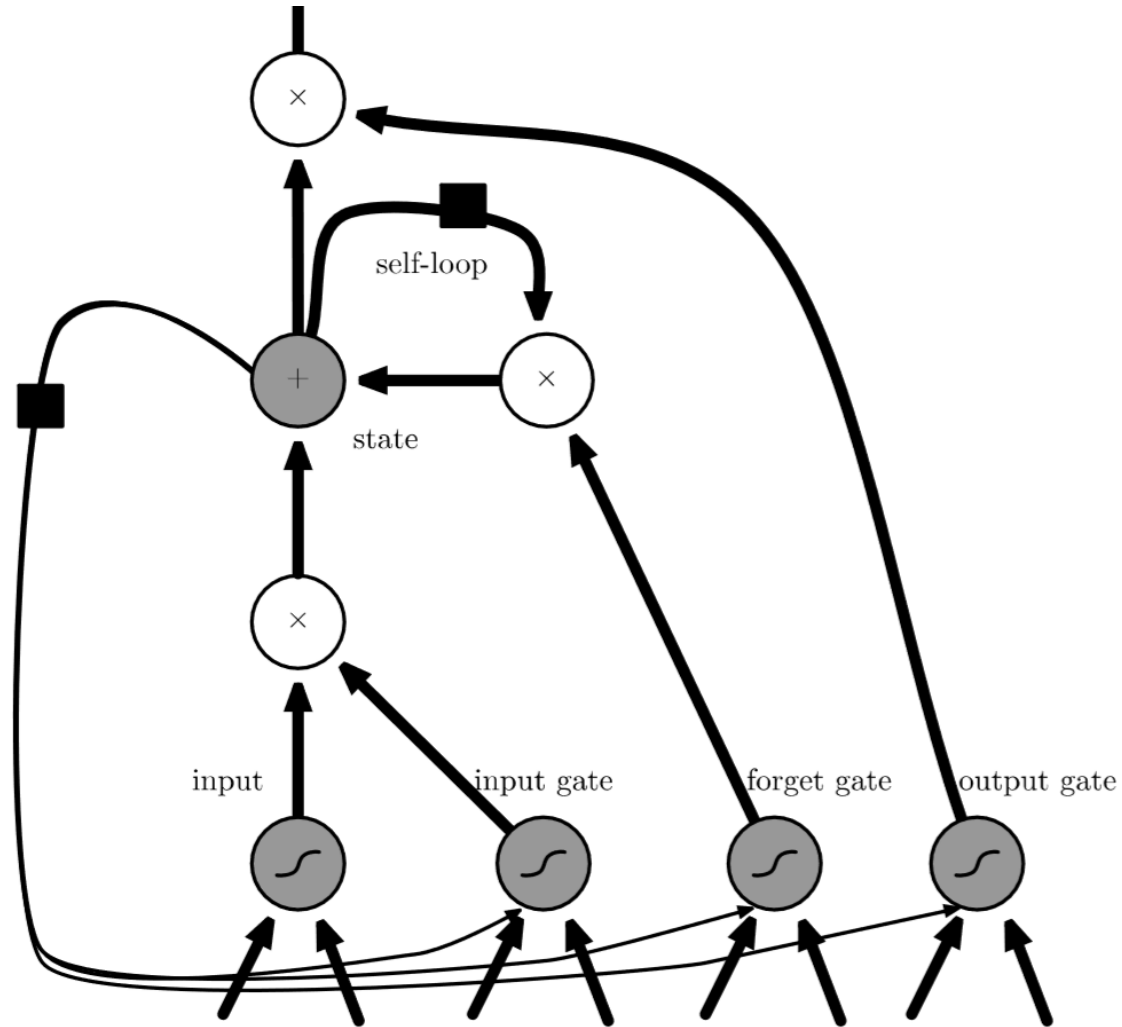
Figure 4.1: **The vanishing gradient problem for RNNs.** The shading of the nodes in the unfolded network indicates their sensitivity to the inputs at time one (the darker the shade, the greater the sensitivity). The sensitivity decays over time as new inputs overwrite the activations of the hidden layer, and the network ‘forgets’ the first inputs.

RNNs can cope with recent correlations



But longer term ones are more difficult..





LSTM

(Long Short **Term** Memory)

GRU

(Gated Recurrent Unit)

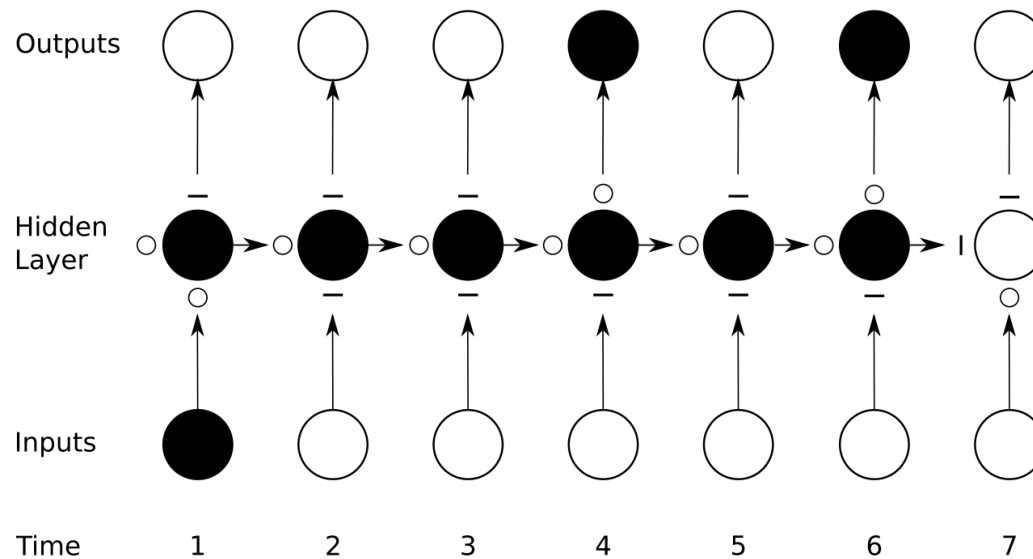
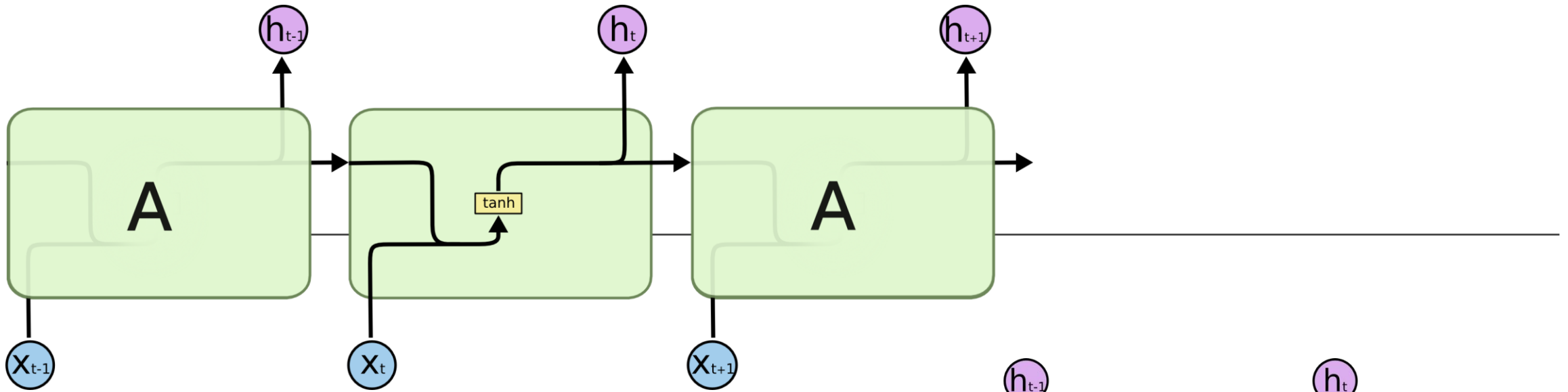
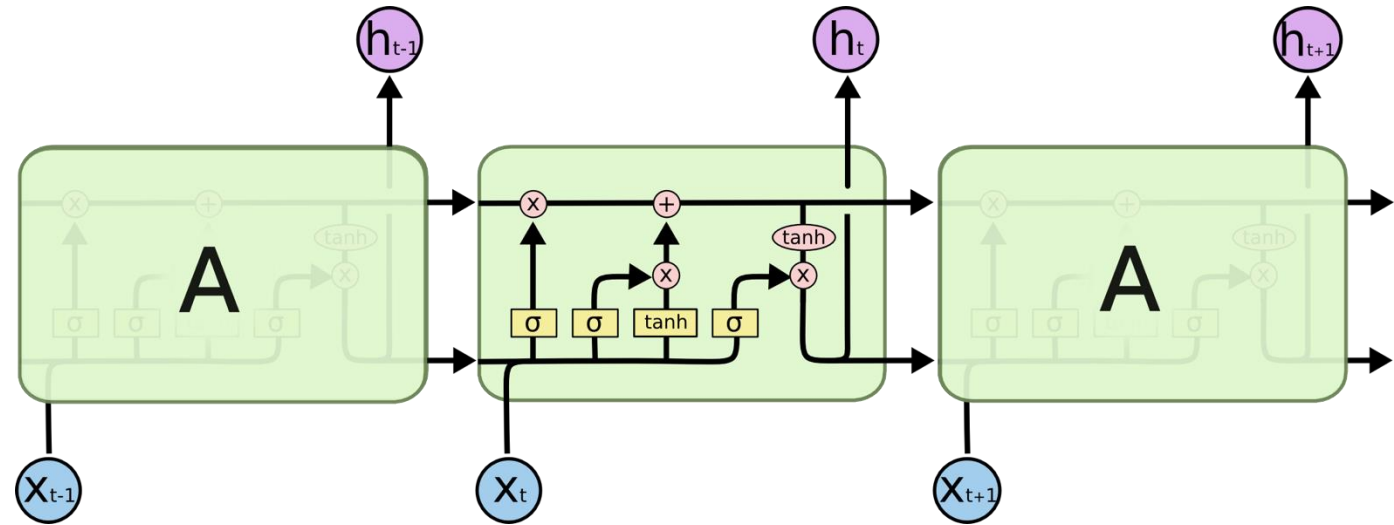


Figure 4.4: **Preservation of gradient information by LSTM.** As in Figure 4.1 the shading of the nodes indicates their sensitivity to the inputs at time one; in this case the black nodes are maximally sensitive and the white nodes are entirely insensitive. The state of the input, forget, and output gates are displayed below, to the left and above the hidden layer respectively. For simplicity, all gates are either entirely open ('O') or closed ('—'). The memory cell 'remembers' the first input as long as the forget gate is open and the input gate is closed. The sensitivity of the output layer can be switched on and off by the output gate without affecting the cell.

Gates can
be used to
preserve
information

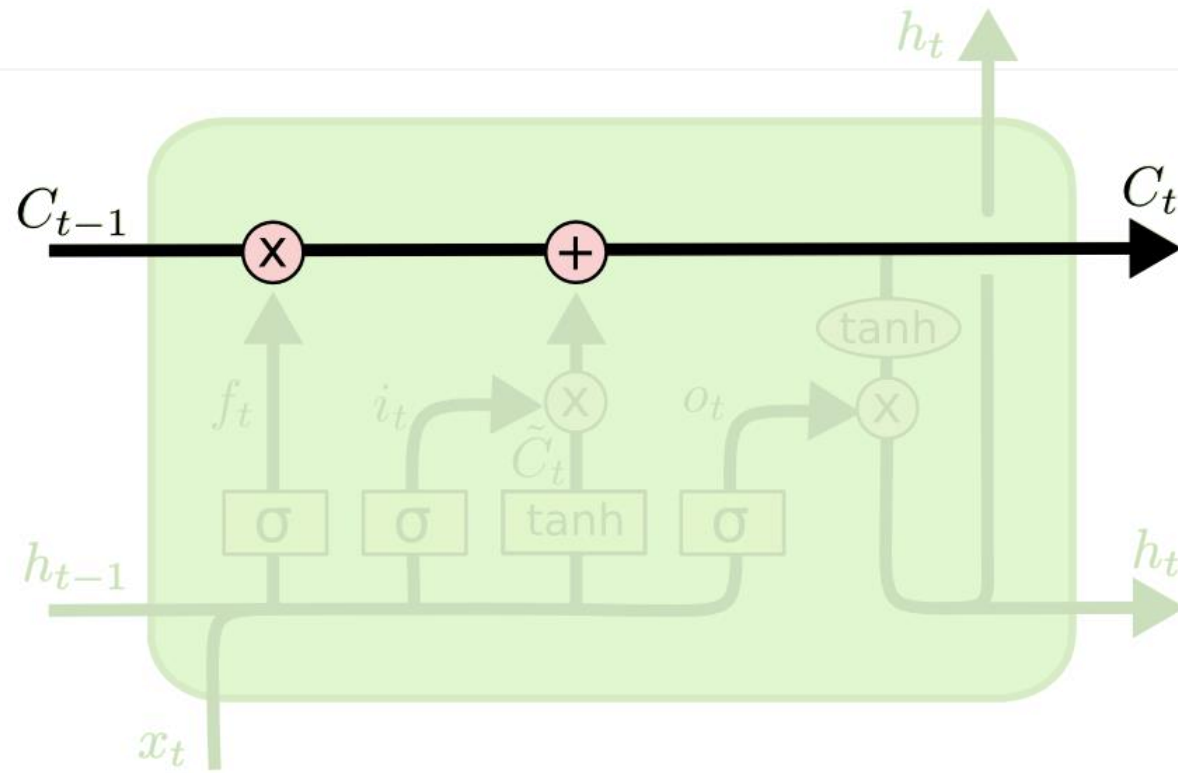


The repeating module in a standard RNN contains a single layer.

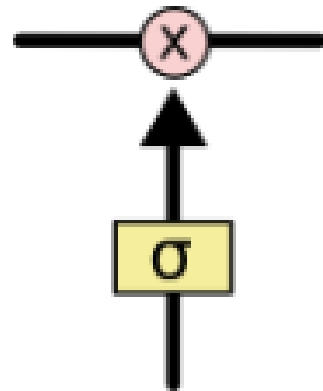


The repeating module in an LSTM contains four interacting layers.

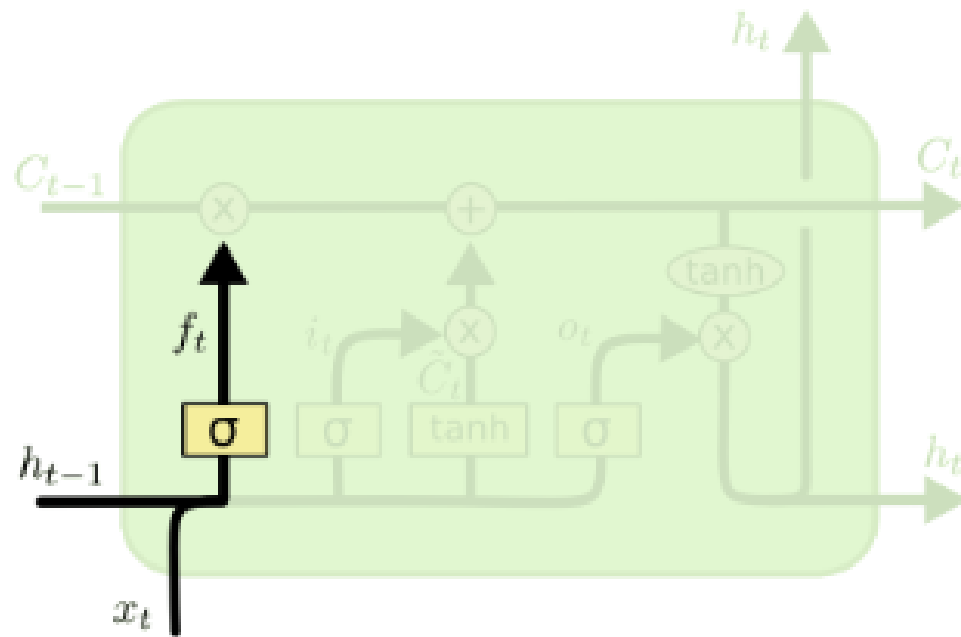
Cell state



Gates

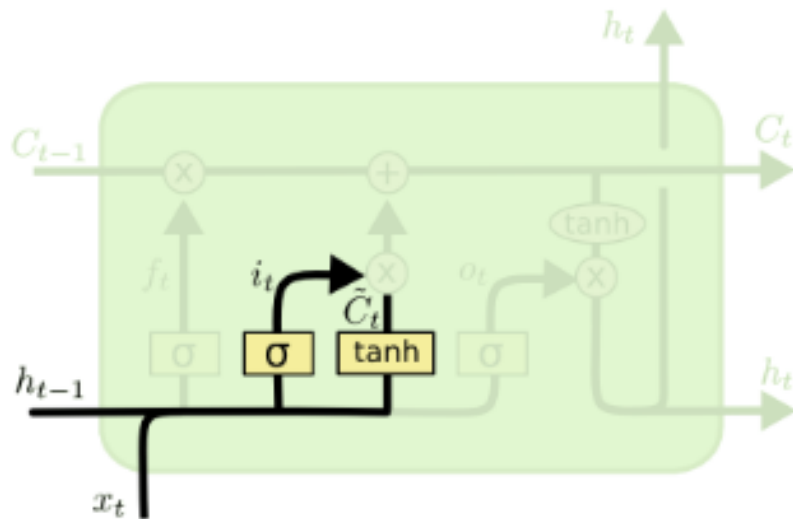


Forget gate ...



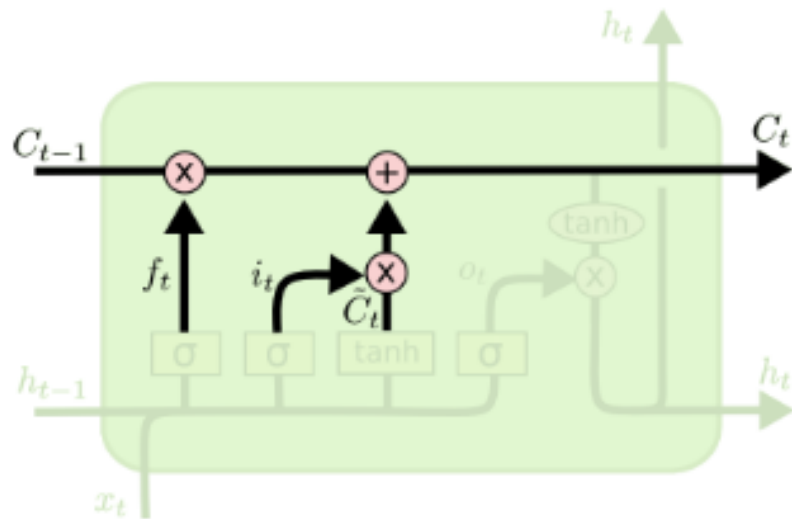
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate ...



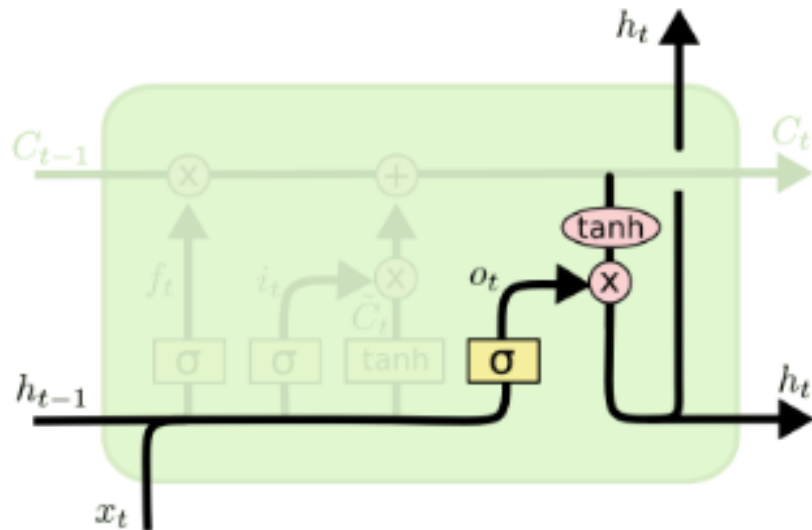
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Updating of cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output gate ...



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Summary of LSTM details

Input, forget, output gates:

$$\mathbf{i}_t = \text{sigm}(\mathbf{b}_{[i]} + \mathbf{U}_{[i]}\mathbf{h}_{t-1} + \mathbf{W}_{[i]}C(w_t))$$

$$\mathbf{f}_t = \text{sigm}(\mathbf{b}_{[f]} + \mathbf{U}_{[f]}\mathbf{h}_{t-1} + \mathbf{W}_{[f]}C(w_t))$$

$$\mathbf{o}_t = \text{sigm}(\mathbf{b}_{[o]} + \mathbf{U}_{[o]}\mathbf{h}_{t-1} + \mathbf{W}_{[o]}C(w_t))$$

The **gates** control the flow of information in $(\mathbf{i}_t, \mathbf{f}_t)$ and out $(\mathbf{o}_t)$ of the cell

Cell state:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{b}_{[c]} + \mathbf{U}_{[c]}\mathbf{h}_{t-1} + \mathbf{W}_{[c]}C(w_t))$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

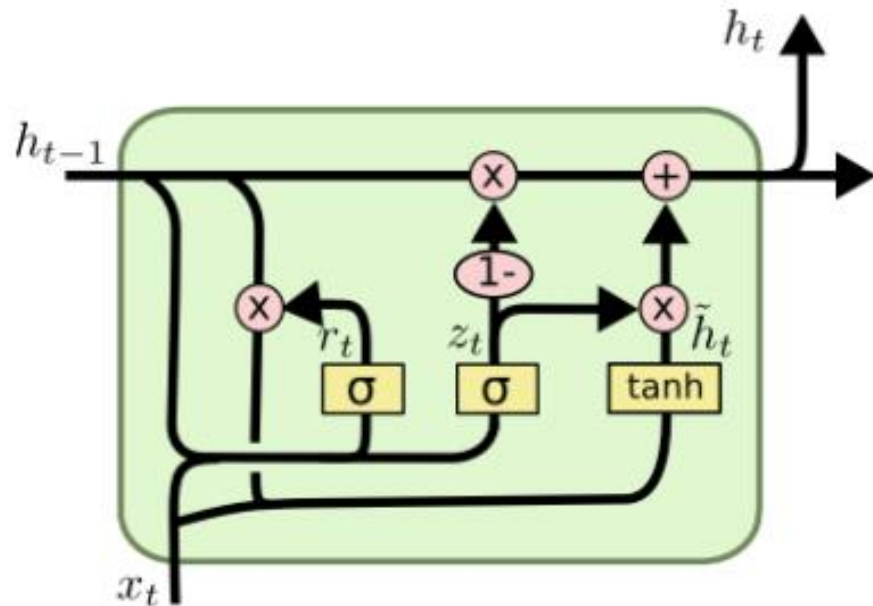
The **cell state** maintains information on the input

Hidden layer:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

The **hidden layer** sees what passes through the output gate

Gated Recurrent Units (GRU)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Summary of GRU equations

Update, reset gates:

$$\mathbf{z}_t = \text{sigm}(\mathbf{b}_{[z]} + \mathbf{U}_{[z]}\mathbf{h}_{t-1} + \mathbf{W}_{[z]}C(w_t))$$

$$\mathbf{r}_t = \text{sigm}(\mathbf{b}_{[r]} + \mathbf{U}_{[r]}\mathbf{h}_{t-1} + \mathbf{W}_{[r]}C(w_t))$$

Fewer **gates**, thus fewer parameters and computations

Cell state:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{b}_{[c]} + \mathbf{U}_{[c]}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{W}_{[c]}C(w_t))$$

$$\mathbf{c}_t = (1 - \mathbf{z}_t) \odot \mathbf{c}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{c}}_t$$

Update gate within **cell update**
Coupling of forget and input gates

Hidden layer:

$$\mathbf{h}_t = \mathbf{c}_t$$

Hidden layer is the cell state,
so fewer computations there too

Comparison

LSTM

Input, forget, output gates:

$$\begin{aligned}\mathbf{i}_t &= \text{sigm}(\mathbf{b}_{[i]} + \mathbf{U}_{[i]}\mathbf{h}_{t-1} + \mathbf{W}_{[i]}C(w_t)) \\ \mathbf{f}_t &= \text{sigm}(\mathbf{b}_{[f]} + \mathbf{U}_{[f]}\mathbf{h}_{t-1} + \mathbf{W}_{[f]}C(w_t)) \\ \mathbf{o}_t &= \text{sigm}(\mathbf{b}_{[o]} + \mathbf{U}_{[o]}\mathbf{h}_{t-1} + \mathbf{W}_{[o]}C(w_t))\end{aligned}$$

Cell state:

$$\begin{aligned}\tilde{\mathbf{c}}_t &= \tanh(\mathbf{b}_{[c]} + \mathbf{U}_{[c]}\mathbf{h}_{t-1} + \mathbf{W}_{[c]}C(w_t)) \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t\end{aligned}$$

Hidden layer:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

GRU

Update, reset gates:

$$\begin{aligned}\mathbf{z}_t &= \text{sigm}(\mathbf{b}_{[z]} + \mathbf{U}_{[z]}\mathbf{h}_{t-1} + \mathbf{W}_{[z]}C(w_t)) \\ \mathbf{r}_t &= \text{sigm}(\mathbf{b}_{[r]} + \mathbf{U}_{[r]}\mathbf{h}_{t-1} + \mathbf{W}_{[r]}C(w_t))\end{aligned}$$

Cell state:

$$\begin{aligned}\tilde{\mathbf{c}}_t &= \tanh(\mathbf{b}_{[c]} + \mathbf{U}_{[c]}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{W}_{[c]}C(w_t)) \\ \mathbf{c}_t &= (1 - \mathbf{z}_t) \odot \mathbf{c}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{c}}_t\end{aligned}$$

Hidden layer:

$$\mathbf{h}_t = \mathbf{c}_t$$

Summary

- RNN are used for processing sequences which may vary in length:
 - Sequence -> Sequence (simultaneous and encoding/decoding)
 - Sequence -> Vector
 - Vector -> Sequence
- The recurrent unit is unwrapped as “time steps” progress
- Since the same unit is repeated ...
 - they are prone to vanishing and exploding gradients.
- Gated cells such as LSTM and GRU can help preserve long-term information in RNNs and make them more useful.

Reading

Karpathy's simple blog intro to RNNs

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Read <http://www.deeplearningbook.org/contents/rnn.html>

- Sections 10.1-10.2, 10.2.2, 10.2.4, 10.3–10.7, 10.10–10.11.1 are examinable

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

On the difficulty of training Recurrent Neural Networks

<https://arxiv.org/pdf/1211.5063.pdf>